Introduktion till C++



Uppgift 7 – FixedVector

Uppgiftens målsättning

Att vi ska utveckla våran egen ”hybrid” mellan `std::array` och `std::vector`.

Uppgift

Till den här uppgiften ska ni skapa följande templatiserade klass:

- FixedVector.

FixedVector

Denna klass påminner om std:s vectorklass, med undantag för att all **data ligger i klassen istället**. Till skillnad från `std::array` så håller den reda på hur många element som är tillagda, och försöker man accessa ett index utan att det är tillagt så kraschar det i debug (med assert).

Klassen ska heta FixedVector och ska vara en template klass som tar in tre argument:

1. Typen på datan som den lagrar.
2. Maxantal element som kan lagras.
3. Dataformat för vilket format den kommer lagra antalet i (antal element) i.

Så här ska interfacet för FixedVector se ut. Du måste själva implementera all funktionalitet. Det är också OK att lägga till fler funktioner.

```
/// <summary>
/// Implements a class with similar interface to std::vector, but stores data inlined in the class with a fixed storage capacity.
/// </summary>
/// <typeparam name="Type"></typeparam>
/// <typeparam name="SizeType"></typeparam>
/// <typeparam name="CAPACITY"></typeparam>
template <typename Type, unsigned int CAPACITY, typename SizeType = unsigned int>
class FixedVector
{
public:
    typedef Type* iterator;
    typedef const Type* const_iterator;

    FixedVector();
    FixedVector(const FixedVector& aFixedVector);
    ~FixedVector();
    FixedVector& operator=(const FixedVector& aFixedVector);

    Type& front();
    const Type& front() const;
    Type& back();
    const Type& back() const;

    Type& at(SizeType aIndex);
    const Type& at(SizeType aIndex) const;

    const Type& operator[](SizeType aIndex) const;
    Type& operator[](SizeType aIndex);

    void push_back(const Type& aObject);
    void insert(iterator aIterator, const Type& aObject);
    void erase(iterator aIterator);
    void pop_back();

    iterator begin();
    const_iterator begin() const;
    const_iterator cbegin() const;

    iterator end();
    const_iterator end() const;
    const_iterator cend() const;

    void clear();
    void resize(SizeType newSize);
    void resize(SizeType newSize, const Type& value);

    bool empty() const;
    SizeType size() const;
```

```
    SizeType capacity() const;

private:
    SizeType mySize = 0;

    Type myStorage[CAPACITY]; //<- for G
    //alignas(Type) char myStorage[sizeof(Type) * CAPACITY]; // <- needed for VG
};
```

Funktionerna ska bete sig som motsvarande funktioner i std::vector, med undantag för när kapaciteten är för liten. Då ska istället FixedVector misslyckas med en assert.

För att underlätta implementationen så får ni ett enhetstestprojekt. Det testar den mesta av funktionaliteten, men inte riktigt allt. För godkänt ska samtliga enhetstester för G-delen gå igenom. Lägg gärna till fler tester för att mer fullständigt testa din implementation.

VG-del:

Följande punkter måste implementeras för VG:

- FixedVector ska ej anropa konstruktorer för element som är en del av capacity, men inte size. T ex: `FixedVector<MyClass, 10>` ska ej anropa några konstruktorer i MyClass. För att göra detta kommer ni behöva använda placement new.
- För godkänt på VG-delen ska samtliga VG-tester fungera. Kör dem genom att definiera `ENABLE_VG_TESTS`

Bonus:

- Gör en ny version som fungerar korrekt med move för ännu mer utmaning.

Inlämning:

- Behövs ingen Exe-mapp denna gång. Lämna bara in en solution med testerna och FixedVector-implementationen.